

Storage format for encrypted data in Pepsiman

Abraham Sharnoff

Document last updated on 2026-09-19

For converting information to and from binary to store in the file, we will be using ToBinary and FromBinary as defined in [abes_nice_things](#).

1: Information

First we will look at what information we need to actually store.

- The [entries](#) themselves
- The names of those entries
- A 64 byte hash of the correct password¹
- A 2048 bit public [RSA](#) key²

The names of the entries are of course stored as Strings.

For technical reasons, the RSA key is stored in a separate file.

1.a: Entries

Entries contain the password, any number of [fields](#) and the rule on clipboard allowance, which is currently not used.

The password is a [String](#), obviously.

1.a.a: Field

Fields contain a [String](#) for the name of the field as well as a [String](#) for the value.

1.a.b: Clipboard rule

This will be one of the following:

Allow Allow copying without clearing the clipboard.

Deny Deny copying

Timer (time) Allow copying but clear the clipboard after the given time has passed

2: The unencrypted format

64 byte hash: [u8; 64]
Number of entries: usize
Names: [String] ³
Entries: [Entry]

Notably the number of names and the number of entries will always be the same because the names are the names of the entries.

¹Created using sha256

²Used to get the required data length for the [key](#) and [IV](#)

³This format is for a slice of a given type, which is like an array in that it is all next to each other and in order but does not store the length

2.a: Entry

Password: String
fields: Vec (String , String)
Clipboard rule: ClipboardRule

2.a.a: ClipboardRule

enum { Allow: 0_u8
Deny: 1_u8
Timer: [2_u8, [Duration](#)]

3: Encryption

The hash is left unencrypted because it needs to be checked against before decryption.

The names are encrypted using the calculated key and the base IV.

The entries are each encrypted using their own IV which is determined by their index. Essentially take the base IV and run it through a modified [sha256](#) which takes every other byte a number of times equal to its index + 1.

So for the first entry you run it once, for the second you run it twice and so on. Every time a section finishes its own encryption, it does get padding added if needed sourced through /dev/urandom. If it ended in such a way that padding is not needed to create a complete block then it is not padded.

Glossary

Key The 240 byte key used for [AES-256](#)

IV The 16 byte initialization vector which is used to block chain [AES](#)